

```

• os.path.join(os.path.dirname(__file__), 'tem-
plates'),
• )

```

Step 3: Modify Template Context Processors

Django uses context processors when processing web requests. There are already several context processors present, but an additional processor is needed to enable parts of the functionality. For this, add the following code at the end of the settings.py file:

```

• # Custom template context processor setting
• TEMPLATE_CONTEXT_PROCESSORS = (
•     "django.contrib.auth.context_processors.auth",
•     "django.core.context_processors.debug",
•     "django.core.context_processors.i18n",
•     "django.core.context_processors.media",
•     "django.core.context_processors.static",
•     "django.core.context_processors.tz",
•     "django.contrib.messages.context_processors.mes-
sages",
•     'django.core.context_processors.request',
• )

```

Step 4: Commit Changes

Apply this last step to execute these commands to commit changes in Git.

```

• # add files
• (venv)$ git add.
•
• # commit files
• (venv)$ git commit -m "set template settings"

```

Conclusion

Django is surely different from other frameworks in the way it is structured and how it performs the tasks. Django is widely preferred for the way it logically performs activities.

In this chapter, we tried to cover the three fundamentals of web development with Django.

- i. Models are used for the data transmission between the database and view, and they contain data definition of a website or an application.